

LECTURE 06

Analyzing Non-Recursive Algorithms

RAM model • input size • $C(n)$ basic-operation counts • best / worst / average case • nested-loop summations

INPUT SIZE

What does n measure?

BASIC OP

What repeats?

COUNT

How many times?

CASES

Best • worst • average

COUNT FIRST

Big-O comes in Lecture 7.

What should you be able to do by the end?

Lecture 6 follows the CDF topic “Analyzing Non-Recursive Algorithms.”

MODEL

Explain the simplified RAM model and what “unit cost” means.

ANALYZE

Identify the operations performed by a non-recursive algorithm.

COUNT

Count a basic operation and derive an exact function of input size.

DISTINGUISH

Separate best-case, worst-case and average-case behavior.

RELATE

Explain how input arrangement or other characteristics change work.

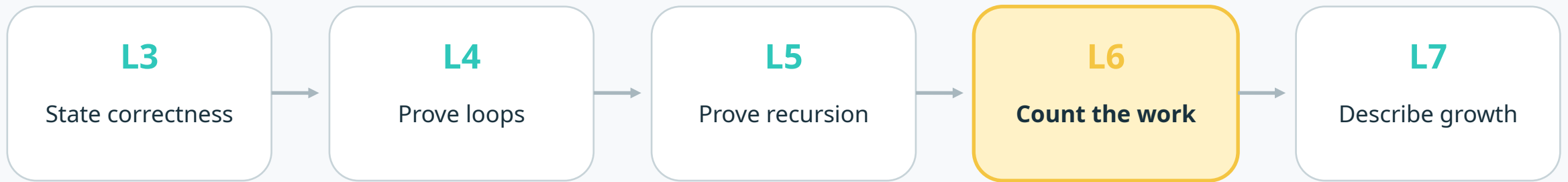
APPLY

Analyze representative algorithms, including iterative MAX(A).

Required CDF example: analyze iterative maximum-finding under different input arrangements.

Where Lecture 6 fits

The course now shifts from proving correctness to measuring efficiency.



Today: exact / concrete operation counts. Next: Big-O, Big-Ω and Θ growth-rate notation.

Correct first. Efficient second.

Efficiency comparison is meaningful only among algorithms that solve the required problem correctly.

1 • CORRECTNESS

Does the algorithm return the required result for every valid input?

2 • ANALYSIS

How many primitive operations does it perform as the input changes?

3 • SELECTION

Among correct choices, which one uses resources more effectively?

A fast wrong answer is still wrong.

Why not just run the program and time it?

Clock time is useful experimentally, but it mixes algorithm quality with hardware and implementation details.

CLOCK TIME DEPENDS ON

- CPU speed
- compiler / language
- cache and memory
- other running processes
- background services

OPERATION COUNT FOCUSES ON

- algorithmic structure
- input size
- repeated work
- input arrangement
- mathematical comparison

OUR GOAL TODAY

Build a machine-independent mathematical description of work before asymptotic notation.

We are not predicting exact seconds; we are modeling how the work changes.

The simplified RAM model

An idealized machine model that makes counting manageable.

SEQUENTIAL

Instructions execute one after another.

UNIT-COST

Simple primitive operations are treated as constant-cost.

MEMORY ACCESS

Reading / writing one memory location is treated as constant-cost.

ABSTRACTION

Ignore CPU, cache, compiler and OS details.

The RAM model is a teaching abstraction — not a literal description of modern hardware.

What counts as a primitive operation?

For introductory analysis, simple statements become constant-cost building blocks.

ASSIGNMENT

$x \leftarrow y$

ARITHMETIC

$x + y, x - y, x \times y$

COMPARISON

$x < y, x = y$

ARRAY ACCESS

$A[i]$

INCREMENT

$i \leftarrow i + 1$

RETURN

return x

We do not need to count every low-level CPU instruction to reason about algorithmic work.

Exact counts depend on what you decide to count

There is no single magical “step count” unless the model is stated.

MODEL A

Count only a representative basic operation, such as the key comparison.

MODEL B

Count assignments, comparisons, increments, array accesses and returns.

IMPORTANT

Both can be valid. The rule is: state the model and use it consistently.

Same algorithm → different exact formula under different cost models

$C(n)$ and $T(n)$: the textbook convention

$C(n)$ counts executions of the chosen basic operation; $T(n)$ estimates running time from that count.

$C(N)$

Count how many times ONE chosen basic operation executes.

SUM: addition executes n times.

$$C(n) = n$$

FULLER STEP COUNT — OPTIONAL

Count several line-by-line events such as initialization, loop-condition checks, body work and return.

$$\text{SUM example: } 1 + (n+1) + n + 1 = 2n + 3$$

This is model-dependent bookkeeping — NOT $C(n)$.

$T(N)$

Estimated running time for the chosen basic operation. If one execution costs c_{op} time units:

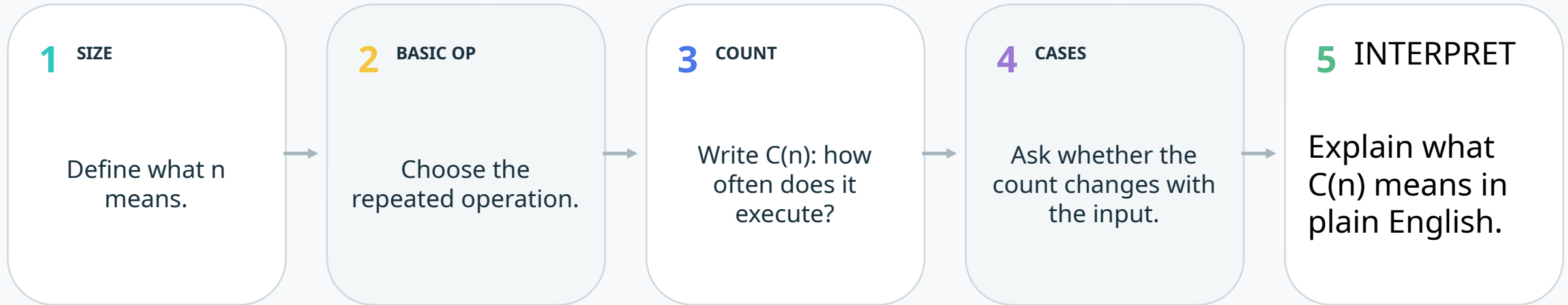
$$T(n) \approx c_{\text{op}} \times C(n)$$

Unlike $C(n)$, the numeric time depends on the machine.

Keep the meanings separate: $C(n)$ = operation count; $T(n)$ = time estimate.

A five-step recipe for non-recursive analysis

Use this sequence every time instead of jumping straight to a formula.



After the one-time $C(n) \rightarrow T(n)$ illustration, the rest of Lecture 6 focuses on deriving $C(n)$.

STEP 1

First identify the input size

The symbol n must mean something concrete for the problem being analyzed.

ARRAY ALGORITHM

n = number of array elements
Example: A has 100 items $\rightarrow n = 100$

STRING ALGORITHM

n = number of characters
Example: "ALGORITHM" has $n = 9$

MATRIX ALGORITHM

Often use two dimensions: m rows and n columns.

Always define n (or m and n) before writing $C(n)$.

The same symbol n can mean different things

Input size belongs to the problem definition, not to the programming language.

Problem	What n means	Example
Find max in A	number of elements	$A=[4,8,1,7] \rightarrow n=4$
Search a string	number of characters	"HELLO" $\rightarrow n=5$
Process graph	often $ V $ and $ E $	10 vertices, 24 edges
Add two n -digit integers	number of digits	$999 + 12 \rightarrow n$ may be 3

Before counting operations, ask: "What exactly grows?"

STEP 2

Then choose a basic operation

$C(n)$ counts how many times this selected basic operation executes.

ALGORITHM

```
SUM(A)
  total ← 0
  for i ← 0 to n-1
    total ← total + A[i]
  return total
```

BASIC OPERATION

Addition: $\text{total} \leftarrow \text{total} + A[i]$ → executes n times.

 $C(N)$

The chosen addition executes once per element: $C(n) = n$

Example: $n=5 \rightarrow C(5)=5$ additions.

The count depends on the number of elements, not on their values.

Keep the focus here on the basic-operation count: $C(n)=n$.

Basic-operation count vs. fuller step count

Both are useful; they answer slightly different questions.

BASIC-OPERATION MODEL

Count one representative repeated operation.

For MAX(A): count $A[i] > \text{largest}$.

Result: $n - 1$ comparisons.

FULLER STEP MODEL

Count initialization, comparisons, increments, updates, return, etc.

For MAX(A): update count depends on order.

Do not compare formulas from two different models as if they were the same measurement.

$C(N) \rightarrow T(N)$

One-time illustration: relate $C(n)$ to a time estimate

We show this relationship once using $SUM(A)$; after this, the lecture stays with $C(n)$.

View	Meaning	$SUM(A)$
$C(n)$	basic-operation count	n additions
c_add	time/cost of one addition on the chosen machine	a constant for that machine
$T(n) \approx c_add \ C(n)$	running-time estimate from the basic operation	$\approx c_add \ n$
Fuller step count	optional line-by-line bookkeeping	$2n+3$ under the stated model

$$T(n) \approx c_add \ n$$

Remember the idea, then return to $C(n)$: the exact basic-operation count is our main language today.

WORKED EXAMPLE 1

SUM(A): start with the algorithm

We will analyze it slowly before writing any formula.

ALGORITHM

```
SUM(A)
  total  $\leftarrow$  0
  for i  $\leftarrow$  0 to n-1
    total  $\leftarrow$  total + A[i]
  return total
```

INPUT SIZE

n = number of elements in A

BASIC OPERATION

The addition total + A[i]

QUESTION

How many times is that addition executed?

WORKED EXAMPLE 1

Trace SUM(A) on a concrete input

Always understand a small instance before generalizing to n .

$$A = [5, 2, 7, 4] \Rightarrow n = 4$$

Iteration	A[i]	total before	total after
i=0	5	0	5
i=1	2	5	7
i=2	7	7	14
i=3	4	14	18

OBSERVATION

There are 4 elements and exactly 4 additions.

$$n = 4 \rightarrow \text{additions} = 4$$

WORKED EXAMPLE 1

Now count the statements

One-time optional bookkeeping example: count several line-by-line events, but do not call this $C(n)$.

Statement / event	Count for n elements
total $\leftarrow$ 0	1
loop condition / evaluation	$n + 1$
total $\leftarrow$ total + A[i]	n
return total	1

BASIC OP COUNT

If addition is the basic operation: $C(n) = n$

FULLER MODEL

Under this stated bookkeeping model:
 $1 + (n+1) + n + 1 = 2n + 3$.

We will not repeat fuller step counting for every example; from here onward, focus on $C(n)$.

WORKED EXAMPLE 1

From the trace to the general formula

The addition occurs once for each element.

1st element

2nd element

...

nth element

1 addition

1 addition

1 addition

$$C(n) = 1 + 1 + \dots + 1 = n$$

English: an array of n elements causes exactly n additions.

For SUM(A), the data values do not change the count

Different values, same n , same number of additions.

Array	n	Basic additions
[1,2,3,4,5]	5	5
[1000,-7,0,2,99]	5	5
[-4,-3,-2,-1,0]	5	5

KEY IDEA

Not every algorithm has different best and worst cases for the chosen basic operation.

WORKED EXAMPLE 2

COUNT-EVEN(A): fixed tests, variable updates

A useful example where one count is fixed but another depends on the data.

ALGORITHM

```
COUNT-EVEN(A)
  count ← 0
  for i ← 0 to n-1
    if A[i] mod 2 = 0
      count ← count + 1
  return count
```

INPUT SIZE

n = number of elements

PARITY TEST

$A[i] \bmod 2 = 0$ executes exactly n times.

COUNT UPDATE

$\text{count} \leftarrow \text{count} + 1$ executes only when the current value is even.

WORKED EXAMPLE 2

Trace COUNT-EVEN on one array

A = [3, 8, 5, 12, 7]

i	A[i]	Even?	count after
0	3	No	0
1	8	Yes	1
2	5	No	1
3	12	Yes	2
4	7	No	2

PARITY TESTS

5 = n

COUNT UPDATES

2 = number of even values

Different operations in the same algorithm can have different count behavior.

What is fixed and what can vary?

For COUNT-EVEN(A), the answer depends on which operation you count.

Operation	Best	Worst	Why?
Parity test	n	n	Every element must be tested
count $\leftarrow$ count + 1	0	n	Depends on how many values are even

LESSON

“Best” and “worst” are meaningless until you say what cost you are measuring.

COUNT-EVEN(A)

One primary $C(n)$, plus a secondary data-dependent count

Let k = number of even values in the array, so $0 \leq k \leq n$.

 $C(N)$

Choose the parity test as the basic operation.

$$C(n)=n$$

WHAT VARIES?

Parity tests are fixed: $C(n)=n$.

The update count varies with the input values: k = number of even elements.

SECONDARY COUNT K

The update count is different from $C(n)$:

k = number of even values
 $0 \leq k \leq n$

It explains data-dependent assignments without redefining $C(n)$.

Main point: $C(n)$ counts the chosen basic operation; other useful counts can be tracked separately.

Best, worst and average case

All three compare inputs of the same size n .

BEST CASE

The least work among all valid inputs of size n .

WORST CASE

The most work among all valid inputs of size n .

AVERAGE CASE

Expected work under a stated probability model for inputs of size n .

Average case is incomplete unless the input distribution / assumption is stated.

Best case does NOT mean “small input”

Case analysis compares arrangements or situations while n stays fixed.

Suppose $n = 1,000,000$.

BEST ARRANGEMENT

Linear search target is at $A[0]$. Only one key comparison is needed.

WORST ARRANGEMENT

Target is last or absent. Up to n comparisons are needed.

Same n . Different work.

WORKED EXAMPLE 3

Linear Search: the algorithm

Here target position directly changes the number of comparisons.

ALGORITHM

```
LINEAR-SEARCH(A, v)
  for i ← 0 to n-1
    if A[i] = v
      return i
  return NOT-FOUND
```

INPUT SIZE

n = number of elements in A

BASIC OPERATION

Key comparison: $A[i] = v$

WHY CASES MATTER

The algorithm may return early, so target position changes the count.

Best case: target is first

$A = [7, 2, 9, 5, 4]$, search $v = 7$

Compare $A[0]$ with 7



MATCH → return immediately

$C_{\text{best}}(n) = 1$

Even when n is huge, the best arrangement needs only one key comparison.

A middle case: target at position 3

$A = [7, 2, 9, 5, 4]$, search $v = 9$

Comparison	Question	Result
1	$A[0] = 9 ?$	No
2	$A[1] = 9 ?$	No
3	$A[2] = 9 ?$	Yes $\rightarrow$ return 2

COUNT

The target at the 3rd position requires 3 key comparisons.

Position $k \rightarrow k$ comparisons

Worst case 1: target is last

A = [7, 2, 9, 5, 4], search v = 4

The algorithm must reject every earlier element before it can find 4.

Checked positions	0	1	2	3	4
Values	7	2	9	5	4
Comparison result	No	No	No	No	Yes

$$C_{\text{worst}}(n) = n$$

Worst case 2: target is absent

A = [7, 2, 9, 5, 4], search v = 8

- A[0] is not 8.
- A[1] is not 8.
- A[2] is not 8.
- A[3] is not 8.
- A[4] is not 8.
- Only after checking all n elements can NOT-FOUND be returned.

WHY THIS MATTERS

“Absent” is often a worst-case input characteristic for search because there is no successful comparison that lets the algorithm stop early.

n comparisons

AVERAGE CASE

Start with a concrete $n = 5$ example

Assume a successful search and each target position is equally likely.

Target position	Comparisons needed
1st	1
2nd	2
3rd	3
4th	4
5th	5

AVERAGE

$$\begin{aligned} & (1 + 2 + 3 + 4 + 5) / 5 \\ &= 15 / 5 \\ &= 3 \text{ comparisons} \end{aligned}$$

Notice: “average” came from an assumption about how likely each position is.

Now generalize from 5 positions to n positions

Successful linear search; target equally likely to occur in any position.

Possible comparison counts:

1, 2, 3, ..., n

Average = sum of possible counts ÷ number of positions

$$C_{avg}(n) = (1 + 2 + \dots + n) / n$$

$$= [n(n + 1)/2] / n = (n + 1)/2$$

If target positions are not equally likely, this average changes.

WORKED EXAMPLE 3

Linear Search — full case comparison

Same algorithm, same n , different input situations.

Case	Situation	Key comparisons
Best	v is at $A[0]$	1
Middle example	v is at position k	k
Worst	v is last or absent	n
Average*	successful; uniform position	$(n + 1)/2$

*Average shown under an explicit probability assumption.

Collect the $C(n)$ cases in one place

Basic operation: compare $A[i]$ with the target. Let k be the successful target position counted from 1.

Quantity	Successful at position k	Target absent
Basic operation	$A[i] = v$	$A[i] = v$
$C(n)$: key comparisons	k	n
$T(n) \approx c_{\text{cmp}} C(n)$	$\approx c_{\text{cmp}} k$	$\approx c_{\text{cmp}} n$

BEST

$C_{\text{best}}=1$

WORST

$C_{\text{worst}}=n$

AVERAGE SUCCESS

$E[k]=(n+1)/2$

Iterative MAX(A): start with the algorithm

We already know what it should return. Now we count its work.

ALGORITHM

```
MAX(A)
  largest ← A[0]
  for i ← 1 to n-1
    if A[i] > largest
      largest ← A[i]
  return largest
```

INPUT SIZE

n = number of elements in A

KEY COMPARISON

$A[i] > \text{largest}$

QUESTION

Does array order change how many key comparisons occur?

Key comparisons are fixed

After A[0] initializes largest, every remaining element is compared exactly once.

Element	Role	Key comparisons
A[0]	initialization	0 comparisons
A[1]	candidate	1 comparison
A[2]	candidate	1 comparison
...	...	...
A[n−1]	candidate	1 comparison

$C(n) = n - 1$ comparisons

Descending input: no updates after initialization

A = [9, 7, 5, 3, 1]

Candidate	Compare with largest	Update?	largest after
start	—	initialize	9
7	7 > 9 ?	No	9
5	5 > 9 ?	No	9
3	3 > 9 ?	No	9
1	1 > 9 ?	No	9

COMPARISONS

4 = n - 1

UPDATES

38

Ascending input: every candidate creates a new maximum

A = [1, 3, 5, 7, 9]

Candidate	Compare with largest	Update?	largest after
start	—	initialize	1
3	3 > 1 ?	Yes	3
5	5 > 3 ?	Yes	5
7	7 > 5 ?	Yes	7
9	9 > 7 ?	Yes	9

COMPARISONS

4 = n - 1

UPDATES

4 = n - 1

Mixed input: updates occur only at “record highs”

A = [4, 9, 2, 11, 6]

Candidate	Compare with largest	Update?	largest after
start	—	initialize	4
9	9 > 4 ?	Yes	9
2	2 > 9 ?	No	9
11	11 > 9 ?	Yes	11
6	6 > 11 ?	No	11

COMPARISONS

4 = n - 1

UPDATES

40

Same n, same comparisons, different updates

This is exactly why we must state what we are counting.

Input	Comparisons	Updates to largest	Result
[9,7,5,3,1]	4	0	9
[1,3,5,7,9]	4	4	9
[4,9,2,11,6]	4	2	11

INTERPRETATION

If the basic operation is the comparison, best = worst = average = $n - 1$. If updates are also counted, arrangement matters.

The comparison count is fixed, but update count u varies

Let u = number of times a new record maximum is found, where $0 \leq u \leq n-1$.

Quantity	Meaning	Count
$C(n)$	basic key comparisons	$n - 1$
u	updates to largest	0 to $n - 1$
Descending input	no new record maximum	$u = 0$
Ascending input	every candidate is a new record maximum	$u = n - 1$

u is useful secondary information, but it does not replace $C(n)$: the chosen basic operation is still the comparison.

CHECK UNDERSTANDING

Predict MAX(A) before calculating

For each array, give comparisons, updates, and returned maximum.

Input	Comparisons?	Updates?	Return?
[8,6,4,2]			
[2,4,6,8]			
[5,2,9,1]			

HINT

All three arrays have $n = 4$, so key comparisons are fixed at 3. Only the update count changes.

WORKED EXAMPLE 4

Rectangular nested loops

When both loop limits are independent, counts multiply.

ALGORITHM

```
for i ← 1 to n  
  for j ← 1 to n  
    BASIC-OP
```

OUTER LOOP

Runs n times.

INNER LOOP

For each outer iteration, runs n times.

$$C(n) = n \times n = n^2$$

WORKED EXAMPLE 4

See n^2 concretely before memorizing it

Let $n = 4$.

Outer i	Inner j values	BASIC-OP executions
1	1,2,3,4	4
2	1,2,3,4	4
3	1,2,3,4	4
4	1,2,3,4	4

TOTAL

$$4 + 4 + 4 + 4 = 16$$

$$\text{And } 4^2 = 16.$$

The “square” comes from n repetitions of n work, not from a magic complexity rule.

Why sigma becomes useful for nested loops

For a single loop, direct counting is easy. With two or three nested loops, sigma mirrors the loop structure.

ONE LOOP

```
for i = 1..n  
  BASIC-OP
```

$C(n)=n$

Sigma is possible, but not necessary.

TWO LOOPS

```
for i = 1..n  
  for j = 1..n  
    BASIC-OP
```

Double sigma cleanly counts both loops.

THREE LOOPS

Three nested loops naturally lead to three summation levels when the basic operation is in the innermost loop.

Rule: sigma is simply a compact way to ADD the number of basic-operation executions.

RECTANGULAR LOOPS

Derive $C(n)$ with a double sum

Choose BASIC-OP inside the inner loop. Double sigma mirrors the two nested loops.

 $C(N)$

BASIC-OP executes n times for each of n outer iterations.

$$C(n) = n^2$$

SIGMA FORM

$$C(n) = \sum_{i=1}^n \sum_{j=1}^n 1$$

$$= \sum_{i=1}^n n$$

$$= n^2$$

ENGLISH INTERPRETATION

For every one of the n outer iterations, the inner loop performs n basic operations.

That gives n groups of n work.

The exact count n^2 comes directly from the loop structure; asymptotic notation waits for Lecture 7.

Matrix addition: a real double-loop example

Add corresponding entries of two $m \times n$ matrices.

ALGORITHM

```
MATRIX-ADD(A,B)
for i ← 0 to m-1
  for j ← 0 to n-1
    C[i][j] ← A[i][j] + B[i][j]
return C
```

INPUT SIZE

Rows = m
Columns = n

BASIC OP

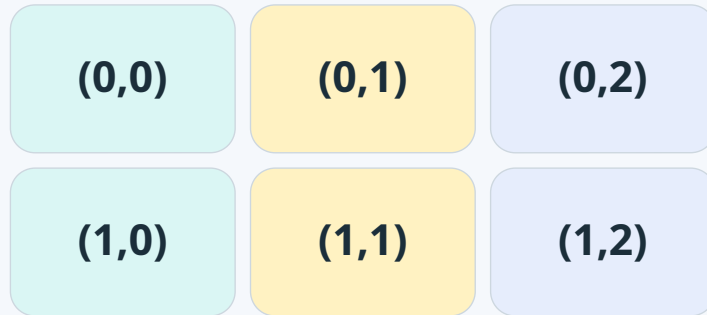
$A[i][j] + B[i][j]$
One scalar addition

QUESTION

How many scalar additions are performed over all matrix positions?

First count a 2×3 example

Every matrix position performs exactly one scalar addition.



ROW VIEW

Row 0 $\rightarrow$ 3 additions

Row 1 $\rightarrow$ 3 additions

Total = 6

$C(m,n)$

$C(2,3)=6$

In general: m rows $\times$
 n columns

Concrete count first; sigma comes next.

Now derive $C(m,n)$ with double sigma

The inner sum counts columns for one row; the outer sum repeats that work across all rows.

INNER SUM

For one fixed row i :

$$\sum_{j=0}^{n-1} 1 = n$$

There are n additions in that row.

OUTER SUM

There are m rows:

$$\sum_{i=0}^{m-1} n = mn$$

Add n , m times.

FINAL $C(m,n)$

$$C(m,n) = \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} 1$$

= mn additions

Double sigma is just the compact form of $n + n + \dots + n$, repeated m times.

Square matrices: set $m = n$

The general $m \times n$ result immediately gives the familiar n^2 count for $n \times n$ matrices.

RECTANGULAR $m \times n$

$$C(m,n)=mn$$

Example: 3×5

$C(3,5)=15$ additions

SQUARE $n \times n$

Set $m=n$:

$$C(n)=n \cdot n = n^2$$

Example: $4 \times 4 \rightarrow 16$ additions

Same algorithm; only the input-dimension notation changes.

Three nested loops: choose scalar multiplication as the basic operation

An optional extension showing how $C(n)$ scales naturally from double to triple nesting.

ALGORITHM

```
MATRIX-MULTIPLY(A,B)
for i ← 0 to n-1
  for j ← 0 to n-1
    C[i][j] ← 0
    for k ← 0 to n-1
      C[i][j] ← C[i][j] + A[i][k] ×
B[k][j]
```

BASIC OP

Scalar multiplication:
 $A[i][k] \times B[k][j]$

LOOP COUNTS

n choices for i
× n choices for j
× n choices for k

Triple sigma gives $C(n)=n^3$ scalar multiplications

Evaluate from the innermost k loop outward.

INNER k LOOP

$$\sum_{k=0}^{n-1} 1 = n$$

For one fixed (i,j), there are n multiplications.

MIDDLE j LOOP

$$\sum_{j=0}^{n-1} n = n^2$$

There are n output columns per row.

OUTER i LOOP

$$\sum_{i=0}^{n-1} n^2 = n^3$$

$C(n)=n^3$ multiplications.

$$C(n)=\sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1 = n^3$$

Sigma for unequal inner-loop lengths

Sigma notation is just shorthand for addition.

$$\sum_{i=1}^{n-1} i$$

means

$$1 + 2 + 3 + \dots + (n - 1)$$

READ THE SYMBOLS

Σ = "sum"

i = running index

1 = starting value

$n - 1$ = ending value

CLOSED FORM

$$1 + 2 + \dots + (n - 1)$$

$$= n(n - 1)/2$$

Understand the expanded sum first; use sigma only as compact notation.

WORKED EXAMPLE 5

Triangular nested loops

Here the inner loop gets longer as i grows.

ALGORITHM

```
for  $i \leftarrow 1$  to  $n-1$   
  for  $j \leftarrow 0$  to  $i-1$   
    BASIC-OP
```

WHEN $I = 1$

inner loop executes 1 time

WHEN $I = 2$

inner loop executes 2 times

... WHEN $I = N-1$

inner loop executes $n-1$ times

Work it out for $n = 5$ first

Then the general formula becomes obvious.

i	Inner executions
1	1
2	2
3	3
4	4

$$\text{Total} = 1 + 2 + 3 + 4 = 10$$

For general n :

$$1 + 2 + \dots + (n - 1)$$

$$= n(n - 1) / 2$$

TRIANGULAR LOOPS

Derive $C(n)$: add unequal inner-loop lengths

Let $Q = 1+2+\dots+(n-1) = n(n-1)/2$.

$C(N)$

BASIC-OP executes Q times.

$$\begin{aligned} C(n) &= Q \\ &= n(n-1)/2 \end{aligned}$$

SIGMA FORM

$$\begin{aligned} C(n) &= \sum_{i=1}^{n-1} i \\ &= 1+2+\dots+(n-1) \\ &= n(n-1)/2 \end{aligned}$$

ENGLISH INTERPRETATION

The inner-loop work changes with i :
 $1, 2, 3, \dots, n-1$.

So we add those counts rather than multiplying n by n .

Focus here on the exact $C(n)$. Lecture 7 will formalize how this expression grows.

WORKED EXAMPLE 6

Count all unordered pairs in an array

This pattern appears later in brute-force algorithms.

ALGORITHM

```
for i ← 0 to n-2  
  for j ← i+1 to n-1  
    process pair (A[i], A[j])
```

WHEN I = 0 $n - 1$ pairs**WHEN I = 1** $n - 2$ pairs**LAST I = N-2**

1 pair

$$C(n) = (n-1) + (n-2) + \dots + 1$$

UNORDERED PAIRS

List the pairs when $n = 5$

 $A = [A_0, A_1, A_2, A_3, A_4]$ (A_0, A_1) (A_0, A_2) (A_0, A_3) (A_0, A_4) (A_1, A_2) (A_1, A_3) (A_1, A_4) (A_2, A_3) (A_2, A_4) (A_3, A_4)

Number of pairs = 10

$$5(5 - 1)/2 = 10$$

UNORDERED PAIRS

Derive the pair-processing $C(n)$

The same triangular count appears because each unordered pair is processed exactly once.

$$P(n) = n(n-1)/2 \text{ pairs}$$

 $C(n)$

If “process pair” is the basic operation:

$$C(n) = P(n) = n(n-1)/2$$

SIGMA FORM

$$C(n) = \sum_{i=0}^{n-2} (n-1-i)$$

$$= (n-1) + (n-2) + \dots + 1$$

$$= n(n-1)/2$$

WHY THIS COUNT?

Each unordered pair is processed exactly once.

The number of pairs itself determines $C(n)$, independent of the element values.

The triangular loop structure is the same counting pattern as choosing every unordered pair exactly once.

Apply the recipe to FIND-MIN

Same structure as MAX — use it to check whether students can transfer the method.

ALGORITHM

```
MIN(A)
  smallest ← A[0]
  for i ← 1 to n-1
    if A[i] < smallest
      smallest ← A[i]
  return smallest
```

INPUT SIZE

n = number of elements

BASIC OP

$A[i] < \text{smallest}$

COUNT

Exactly $n - 1$ key comparisons for every arrangement.

Update count varies with order, just as it did for MAX.

FIND-MIN

C(n) for FIND-MIN

The selected basic operation is the key comparison $A[i] < \text{smallest}$.

C(N)

Choose $A[i] < \text{smallest}$ as the basic operation.

$$C(n) = n - 1$$

SIGMA VIEW

$$C(n) = \sum_{i=1}^{n-1} 1$$

= $n - 1$ comparisons

Same for every arrangement.

UPDATE COUNT

The number of assignments to smallest can vary with order.

That is secondary information; it does not change the chosen $C(n) = n - 1$ comparison count.

MIN and MAX share the same analysis structure; only the comparison direction changes.

Compare correct solutions using exact counts

Even before Big-O, operation counting can expose a meaningful difference.

ITERATIVE

```
ITERATIVE-SUM(n)
  S ← 0
  for i ← 1 to n
    S ← S + i
  return S
```

COUNT

n additions in the loop

DIRECT FORMULA

```
DIRECT-SUM(n)
  S ← n(n + 1) / 2
  return S
```

COUNT

A fixed small number of arithmetic operations

Same required result → different operation counts. Correctness remains mandatory.

Use $C(n)$ carefully when the operations differ

The iterative version repeats addition; the direct formula uses several different arithmetic operations.

View	ITERATIVE-SUM	DIRECT-SUM
$C(n)$	n additions (basic op = addition)	No input-dependent repetition; fixed arithmetic count
$T(n)$	$\approx c_{\text{add}} n$	fixed combination of arithmetic-operation costs
Interpretation	work increases with n	work does not increase with n under the RAM model
Growth driver	n	none

This is why the direct formula remains constant-work under the simplified RAM model while the loop grows with n .

Use $C(n)$ on matrix and nested-loop examples

No $T(n)$ here: define the input size, choose the basic operation, and derive its exact count.

A • MATRIX ADD

$m \times n$ matrices

```
for i=0..m-1
  for j=0..n-1
    C[i][j]=A[i][j]+B[i][j]
```

Count additions.

B • SQUARE LOOP

```
for i=1..n
  for j=1..n
    OP
```

Write $C(n)$ directly and with double sigma.

C • TRIANGULAR LOOP

```
for i=1..n-1
  for j=0..i-1
    OP
```

Write the expanded sum, sigma form, and closed form.

Suggested answers: mn , n^2 , and $n(n-1)/2$ respectively.

Your turn — analyze four snippets

State n , choose a basic operation, then count it. Do not write Big-O yet.

A • COUNT POSITIVE

```
for i ← 0 to n-1  
  if A[i] > 0  
    count ← count+1
```

How many positivity tests?

B • FIND TARGET

linear search

Give best, worst and average successful comparisons.

C • ALL PAIRS

```
for i ← 0 to n-2  
  for j ← i+1 to n-1  
    process pair
```

How many pairs?

D • SQUARE LOOP

```
for i ← 1 to n  
  for j ← 1 to n  
    do OP
```

How many OPs?

Work from a small n if you are stuck, then generalize.

CHECK YOURSELF

Possible answers to the class practice

Equivalent reasoning is fine if the cost model is stated clearly.

A • COUNT POSITIVE

Choose positivity test
as basic op.
 $C(n)=n$.
Updates depend on
how many positives
occur.

B • FIND TARGET

Best = 1
Worst = n
Average successful =
 $(n+1)/2$ under uniform
position.

C • ALL PAIRS

$(n-1)+(n-2)+\dots+1$
 $= n(n-1)/2$

D • SQUARE LOOP

n outer iterations $\times$ n inner
iterations = n^2 basic
operations.

Separate fixed counts from data-dependent counts whenever useful.

Five analysis mistakes to avoid

Most errors happen before the algebra.

- 1 Never defining what n means.
- 2 Counting different operations in different algorithms without saying so.
- 3 Calling an input “best” because n is small.
- 4 Giving an average case without stating a probability assumption.
- 5 Jumping to Big-O before understanding the actual count.

Good analysis begins with a precise model, not with memorized complexity labels.

What you should now be able to do

Lecture 6 is mainly about selecting a basic operation and deriving its exact count $C(n)$.

01 • RAM MODEL

Use an idealized unit-cost model.

02 • INPUT SIZE

Define n precisely.

03 • BASIC OP

Choose repeated work to count.

04 • CASES

Best / worst / average depend on input.

05 • EXACT $C(N)$

Write $C(n)$ using direct counting or sigma notation.

06 • NESTED LOOPS

Use single, double, or triangular sums as the loop structure requires.

Next: Asymptotic Analysis and Growth Rates — Big-O, Big-Ω, Θ and dominant growth.

Course references for this lecture

The deck follows the CDF scope and deliberately stops before asymptotic notation.

PRIMARY COURSE TEXT

Anany Levitin, Introduction to the Design and Analysis of Algorithms, 3rd ed. — Chapter 2: Fundamentals of the Analysis of Algorithm Efficiency.

COURSE SEQUENCE

Lecture 6: exact counting, RAM model, input size, best/worst/average and non-recursive examples. Lecture 7: asymptotic notation and growth rates.

PRESENTATION NOTE

Examples are expanded for teaching clarity: concrete trace → count → general formula → interpretation.